

Logical Induction

How to be sensibly uncertain about mathematics you have not finished computing

Lecture notes, Agent Foundations module
Iliad Intensive

June 2026

Abstract

You can know the rules of arithmetic completely and still not know whether some particular large number is prime, because you have not done the division. You can hold a program's full source code and still not know what it prints, because you have not run it. This is uncertainty of a kind ordinary probability theory cannot represent, because the standard theory quietly assumes that anyone who knows the premises already knows all their consequences. These notes develop logical induction, the framework of Garrabrant and collaborators that gives a bounded reasoner a principled way to be uncertain about such logical and computational facts and to grow less wrong as it thinks longer. They assume no background: probability as a degree of belief, what a Dutch book is, what "efficient computation" means, are all built up here from examples. The central picture is a betting market. Each mathematical statement is a share that will eventually pay one dollar if the statement gets proved and nothing if it gets refuted; the market's price for a share is read as the reasoner's probability that the statement is true; and a slow background process keeps publishing newly-proved theorems, like a news ticker of mathematical facts. We then declare the reasoner rational under a single condition: no *efficiently computable* betting strategy, nothing a reasonably fast program could run, can use the market's prices to make unlimited profit off a limited stake. That one condition, that no fast trader can pump you, turns out to force a remarkable amount of good judgment. The prices settle down to a coherent assignment of probabilities; the reasoner comes to bet correctly on theorems long before they are proved, no matter how hard the proofs; it learns the right frequencies for patterns it cannot crack, respects logical relationships among statements it cannot yet evaluate, never grows dogmatic without proof, and even comes to trust its own future judgments. We build the market and its criterion precisely, indicate why such a market provably exists and can be run by a computer, survey what follows from the criterion, and connect it back to the coherence theorems it generalizes and the problems it leaves open.

Contents

1 The uncertainty that more evidence cannot remove	2
2 What we would want from a bounded reasoner	2
3 The betting market picture	3
4 The logical induction criterion	4
5 How such a market is built	6
6 What the single criterion forces	7
7 Connections and open problems	8
Sources	8

1 The uncertainty that more evidence cannot remove

There are two quite different reasons you might fail to know whether something is true, and ordinary probability theory only handles one of them. Telling them apart is the first step.

The familiar kind is uncertainty about the world, which we will call *empirical* uncertainty. You do not know whether it will rain tomorrow, or what time it is in a city you cannot see, because you are missing observations; gather the right evidence and the uncertainty dissolves. This is the kind of uncertainty that probability theory was built for. The standard recipe, named after Thomas Bayes and called *Bayesian* reasoning, is to hold a set of degrees of belief, one number between zero and one for each possibility (a number of 1 meaning certainty that it is so, 0 certainty that it is not, and values in between meaning intermediate confidence), and to revise those numbers as evidence arrives. A degree of belief of this kind is called a *probability* or a *credence*, and a reasoner who manages uncertainty this way is a Bayesian reasoner.

The other kind of uncertainty is stranger, and it survives even when no observation whatsoever is missing. You know, in complete detail, the source code of a program, and you still do not know what it will output, because you have not run it. You know every rule of arithmetic, and you still do not know whether 19,483 is prime, because you have not carried out the divisions. You know exactly how π is defined, and you still do not know whether its ten-billionth digit is a seven. Confronted with the question of whether

$$1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 \quad \text{is even,}$$

most people have to pause and count the ones before answering, which already shows that knowing what a statement *means* and knowing whether it is *true* are different states of mind, separated here not by any missing fact about the world but by missing *computation*. We will call this *logical* uncertainty: uncertainty about the consequences of things you already fully know, where the bottleneck is that you have not yet done the thinking, and no amount of looking at the world would do the thinking for you.

Ordinary Bayesian probability cannot represent this gap, and the reason is built into its foundations. A Bayesian reasoner's beliefs are degrees of confidence spread over fully-specified possibilities, and a fully-specified possibility already settles every mathematical question one way or the other. The framework assumes, in effect, that any computation can be performed instantly and for free, a property called *logical omniscience*. A textbook Bayesian who knows the rules of arithmetic must therefore, to avoid inconsistency, already assign certainty to every theorem of arithmetic and to every falsehood's negation, including theorems no one has proved and falsehoods no one has refuted. That is not a description of any reasoner who has to budget its thinking time, which is to say it is not a description of any real or embedded agent. This is why the topic matters for understanding agents that are part of the world: such an agent cannot hold the full unfolding of its own knowledge in its head, so an account of good reasoning for it must say what it means to be *appropriately unsure about logic itself*, and how to sharpen that unsureness over time as one computes more.

The plan is to first collect the properties we would want from such a reasoner (Section 2), since the field has long agreed more about the wish list than about any way to satisfy it. Section 3 introduces the central image, a betting market over mathematical statements, with every piece of jargon explained as it arrives. Section 4 turns that image into a precise criterion of rationality and states the theorem that such reasoners exist. Section 5 sketches how one is actually built and why a computer can run it. Section 6 is the payoff, the long list of good behaviors that the single criterion forces. Section 7 connects the framework to the coherence theorems it descends from and names the problems it leaves open. Nothing beyond elementary probability and the idea of a computer program is assumed.

2 What we would want from a bounded reasoner

Because it is not even obvious what would count as solving logical uncertainty, the subject has proceeded by listing properties a good reasoner ought to have and looking for tensions among them. Meeting these in your mind first is worthwhile, partly to fix intuitions and partly because the striking fact about logical induction is that one condition delivers essentially all of them together.

Desideratum 1 (Computable). The procedure for assigning probabilities to mathematical claims, and refining them as one thinks longer, should be something a computer can actually carry out. A good reasoner

is never finished, since there is always more to deduce, but the process of refining its beliefs should at least be mechanizable.

Desideratum 2 (Coherent in the limit). The beliefs the reasoner is approaching, the ones it would hold given unlimited time to think, should be logically consistent: certainty for genuine theorems, zero for contradictions, and a higher probability never given to a statement than to anything it logically implies.

Desideratum 3 (Approximately coherent along the way). Consistency only in the infinite limit is too weak; a good bounded reasoner should be roughly consistent at finite times too. If it has noticed that two claims cannot both be true, say “this program outputs 0” and “this program outputs 1,” it should assign them probabilities summing to at most one *even before* it can run the program, rather than holding nonsense until the instant the computation finishes.

Desideratum 4 (Sensible on patterns it cannot crack). With no specific information bearing on a hard question, the reasoner should fall back on the rate at which similar questions come out true. It should put about ten percent on “the n th digit of π is a seven” for a large n it cannot compute, since the digits of π behave, to anyone who cannot predict them, like one fair draw from ten possibilities. A reasoner certain, for no reason, that the trillionth digit is a nine has a broken method.

Desideratum 5 (Not dogmatic). The reasoner should not become certain of a mathematical claim it has not actually settled. Once a Bayesian assigns something probability zero or one it can never revise, so a reasoner that prematurely locks in an extreme belief about an unproved claim has thrown away the ability to change its mind when a proof finally arrives.

These pull against one another. Timeliness pushes toward bold guesses, caution toward hedging; statistical guessing on hard questions sits awkwardly beside strict coherence. Much of the historical difficulty was that proposals could satisfy some of these while violating others. Logical induction’s contribution is a single criterion from which all of them follow as theorems.

3 The betting market picture

The conceptual leap is to stop imagining the reasoner as directly maintaining a table of probabilities and instead to imagine it as running a *betting market*, with the probabilities showing up as market prices. Since not every reader has a picture of how a betting market works, we build it piece by piece; each later definition is a precise version of one of these pieces.

A betting market, in the sense we need, is a place that issues *shares* tied to uncertain claims and stands ready to buy and sell them at posted prices. Here each mathematical statement gets its own share. A share in the statement ϕ is a contract that will pay its holder exactly one dollar if ϕ turns out to be true and nothing if ϕ turns out to be false. The market sets a price for each share and will both buy and sell at that price. If it prices a share in “19,483 is prime” at seventy-five cents, we read that as the reasoner currently judging the statement seventy-five percent likely. The reason a price can be read as a probability is the same no-sure-loss logic that runs through these notes: if a share certain to pay a dollar were on sale for less than a dollar, or a share certain to be worthless were priced above zero, anyone could make free money off the market, so a market nobody can milk for free must price its shares at the true probabilities of the outcomes.

What grounds truth over time is a *deductive process*, which is simply a slowly-growing list of mathematical statements that have been proved, published a bit more each day. Picture a community of mathematicians submitting machine-checked proofs to a shared, ever-lengthening database, or concretely a fixed program that on day n has written down every theorem of arithmetic whose proof is at most n symbols long. This process never publishes anything false and eventually publishes every theorem, but on any given day it has revealed only finitely much. The reasoner’s job is to *run ahead* of it, pricing shares in conjectures that will eventually be proved up toward a dollar, and shares in claims that will eventually be refuted down toward nothing, well before the relevant proofs actually appear in the list.

The last piece is a *trader*: a strategy, run by some program, that watches the history of prices and places buy and sell orders, hunting for shares it thinks are mispriced. The decisive choice in the whole theory is which traders we hold the market accountable to. We do not demand that the market be unbeatable by

every conceivable strategy, including ones that would take longer than the age of the universe to compute, because that demand turns out to be the very logical omniscience we are trying to escape. We demand only that the market be unbeatable by *efficiently computable* traders. A computation is called efficient, or runs in *polynomial time*, if the work it takes grows no faster than some fixed power of the size of its input (the number of steps is at most the input size squared, or cubed, or the like) rather than exploding exponentially; this is the standard dividing line between computations a reasoner could realistically afford to run and ones it could not. The reasoner is declared rational exactly when no affordable, efficiently-computable trader can systematically beat its prices. That sentence is the entire idea, and the next section makes each of its words exact.

4 The logical induction criterion

Here is the definition the whole framework builds toward, followed by careful unpackings of the five notions it rests on.

Definition 4.1 (The logical induction criterion). A market satisfies the *logical induction criterion* relative to a deductive process if no efficiently computable trader can exploit the market relative to that process. A market meeting this criterion is called a *logical inductor*.

Markets, made precise. Write $\mathcal{S}$ for the set of all the statements we might price. A *pricing* assigns to each statement a number between zero and one, computed by some program, with the reading that a share in ϕ is worth a dollar if ϕ is true. A *market* is a sequence of pricings, one for each day, $P_1, P_2, P_3, \dots$, that a program can generate; $P_n(\phi)$ is the price of a ϕ -share on day n . In the version we ultimately build, each day’s pricing is a *belief state*: a finite list of (statement, probability) pairs that the reasoner literally writes down and revises the next day, with every statement not on the list priced at zero. So the mental image is a reasoner filling in one column of an ever-growing spreadsheet per day, each column its complete set of explicit beliefs that day, justified after the fact by the coherence theorem below.

The deductive process, and “worlds.” The deductive process is, formally, a growing sequence of finite sets of statements $D_1 \subseteq D_2 \subseteq \dots$, generated by a program, with D_n the theorems revealed by day n . Against it we set the notion of a *world*: a complete way of assigning “true” or “false” to every statement. The crucial and counterintuitive move, the one that makes the whole theory possible, is that we allow worlds to be logically *wrong*. A bounded reasoner cannot tell at a glance which complete truth-assignments secretly harbor a contradiction, since exposing a contradiction takes work, so we cannot restrict attention to the logically consistent worlds. Instead we use a weaker, mechanically-checkable kind of coherence.

Definition 4.2 (Propositional consistency). A world is *propositionally consistent* if it respects the basic logical connectives “and,” “or,” “not”: it must call “ ϕ and ψ ” true exactly when it calls both ϕ and ψ true, must call “not ϕ ” true exactly when it calls ϕ false, and so on. It need *not* respect deeper mathematical facts; a propositionally consistent world may cheerfully deny a theorem nobody has proved yet, because spotting that the statement is a theorem would take a computation the bookkeeping is not required to do. Given the revealed theorems D , write $\text{PC}(D)$ for the propositionally consistent worlds that make every revealed theorem true.

Propositional consistency is exactly the part of consistency a bounded reasoner can check on the spot, by inspecting the surface logical form of a statement, where full logical consistency would require settling arbitrary mathematics. As the deductive process publishes more theorems, the worlds in $\text{PC}(D)$ that contradicted those theorems get thrown out, so this loose notion of “a world still possible given what has been proved so far” tightens over time toward genuine logical possibility.

Shares, prices, and what a trade is worth. Before defining a trader we need to say exactly what it trades. A *share* of the sentence ϕ is the contract from Section 3: it pays its holder one dollar if ϕ is true and nothing otherwise. The price it changes hands at on day n is $P_n(\phi)$, and we give that price a name as a quantity a trader can look up, the *price feature* ϕ_n^* , defined simply by $\phi_n^* := P_n(\phi)$, the price of ϕ on day

n . Buying one share of ϕ on day n therefore means paying ϕ_n^* now in exchange for a payout of one dollar if ϕ turns out true, so the holding it leaves you with is worth, in honest terms, “one dollar if ϕ , minus the ϕ_n^* you paid.” We will write that holding as the formal combination $\phi - \phi_n^*$: a unit of the sentence ϕ (worth a dollar when ϕ is true) net of the cash ϕ_n^* handed over for it.

Trading strategies, made precise. A trader must decide, each day, how many shares of which sentences to buy or sell, and it is allowed to base that decision on the prices it has seen. To express “how many shares” as a function of the price history we need a class of functions to draw the quantities from; these are the *expressible features*, continuous real-valued functions of the prices built up from price lookups ϕ_n^* , rational constants, and arithmetic (addition, multiplication, and a capped maximum). For instance the expressible feature $\xi := \max(0, \phi_6^* - \psi_7^*)$ checks whether ϕ ’s price on day 6 exceeds ψ ’s price on day 7, returning the gap if so and zero otherwise. A *trading strategy* for day n is then an *affine combination* of sentences with expressible-feature coefficients,

$$T_n = c + \xi_1 \phi_1 + \cdots + \xi_k \phi_k,$$

read as follows. The coefficient on a sentence ϕ_i , written $T_n[\phi_i] = \xi_i$, is the number of shares of ϕ_i the strategy buys that day (a negative number meaning it sells $|\xi_i|$ shares). The bare cash term, written $T_n[1] = c$, is the net number of dollars the strategy takes in (negative meaning it pays out). These are not independent: the cash term is fixed so that the whole bundle costs exactly the going price, $c = -\sum_i \xi_i \phi_{i,n}^*$, which is the precise sense in which a trade is fair at the moment it is made. Equivalently, the strategy is a sum of “buy ξ_i shares of ϕ_i at the prevailing price” holdings, $T_n = \sum_i \xi_i (\phi_i - \phi_{i,n}^*)$, and so has value zero according to the day- n prices themselves. A trader makes money not at the moment of trading but later, when the deductive process reveals which sentences were worth holding.

A concrete day’s trades. Suppose that on some day a trader with a basic grasp of arithmetic, and some skepticism about the market’s overconfident conjectures, sees the prices in the left two columns below and places the orders in the right column:

sentence	price that day	order
$\phi : 1 + 1 = 2$	90¢	buy 4 shares
$\psi : 1 + 1 \neq 2$	5¢	sell 3 shares
χ : Goldbach’s conjecture	98¢	sell 1 share

The net cash these orders move is $4 \cdot 90¢ - 3 \cdot 5¢ - 1 \cdot 98¢ = \2.47 paid out, so for this strategy $T_n[1] = -2.47$, alongside $T_n[\phi] = 4$, $T_n[\psi] = -3$, $T_n[\chi] = -1$. The trader has spent \$2.47 to take a position that bets heavily that $1 + 1 = 2$ (cheaply underpriced at 90¢) and against the market’s near-certainty about Goldbach, and it profits exactly insofar as the eventual proofs vindicate those bets.

Why the coefficients must be continuous. The continuity required of expressible features is not a technicality; it is what makes a fair-priced market findable at all. The market we will build sets its prices in response to how traders trade, while traders trade in response to the prices, a circular dependency. Continuity tames the circle. Imagine a self-referential sentence χ that says “I am true exactly when my own price is below fifty cents.” A trader allowed a discontinuous rule, buy χ whenever its price is below 50¢ and sell it otherwise, could never be cleared: below 50¢ the sentence pays out and buyers get rich, at or above 50¢ it does not and sellers get rich, with no price that leaves the trader content. A continuous rule cannot have such a jump, so between a price where the trader buys and one where it sells there is always a price in between where it neither buys nor sells, and that balancing price is what the market can settle on. Continuity is also what later lets logical inductors sidestep exactly these paradoxes of self-reference.

Traders. A *trader* is a sequence $T = (T_1, T_2, \dots)$ of trading strategies, one per day; picture someone who each morning studies the price history so far and writes down that day’s orders. The trader’s accumulated position after day n is the running sum $\sum_{i \leq n} T_i$ of its daily strategies, again an affine combination of sentences and cash. To ask what that position is worth in a given world W (a complete assignment of true or false to every sentence) we read off its value linearly, treating each held sentence as worth a dollar if W

makes it true: $W(c + a_1\phi_1 + \dots) = c + a_1W(\phi_1) + \dots$, with $W(\phi) = 1$ for true and 0 for false. A trader is *efficiently computable* if the program producing its daily strategy runs in time polynomial in the day number n , so it may think a little longer each day but never an unreasonable amount.

Exploitation, the heart of the matter. This is the precise form of “the trader beats the market.” Follow a trader’s net holdings over time: the shares it owns, minus the cash it has spent. Each propositionally consistent world gives one way of valuing those holdings, counting every owned ϕ -share as worth a dollar if that world calls ϕ true. The collection of all such valuations, over every day and every world still consistent with the theorems revealed so far, is the set of honest appraisals of how the trader is doing.

Definition 4.3 (Exploitation). A trader *exploits* a market if the set of all these plausible appraisals of its net holdings,

$$\left\{ W\left(\sum_{i \leq n} T_i\right) : n \in \mathbb{N}, W \in \text{PC}(D_n) \right\},$$

is bounded below but not bounded above. In words: there is a fixed floor below which the trader’s net worth can never plausibly fall (it never risks more than some finite stake), and yet there is no ceiling on how high its plausible net worth climbs. The trader makes unlimited returns off a limited investment.

A simple case fixes the idea. Suppose a market stubbornly prices “ $1 + 1 = 2$ ” at fifty cents every single day, and a trader buys one such share daily. Before the deductive process gets around to publishing the (trivial) proof that $1 + 1 = 2$, the trader is paying fifty cents a day and its shares might, as far as a still-ignorant world is concerned, be worthless, so its possible loss is capped at the modest total it has spent. But from the day the proof appears onward, every world still in the running must value each of those shares at the full dollar, and the trader, having bought one every day, holds a position worth more and more without limit while having staked only a finite amount. That is exploitation, and a market that permits it is not a logical inductor. (Exploitation is subtler than “buy something that pays out.” A trader can also exploit a market by arbitraging between two statements whose “or” is provable even though neither is ever decided individually, so the criterion rewards spotting logical relationships, not just betting on eventual payouts.)

The criterion, and that such reasoners exist. With all five notions in hand, Definition 4.1 says a market is a logical inductor exactly when no efficiently computable trader can exploit it. The content of the criterion is captured in one sentence worth holding onto, because every property in Section 6 is an instance of it: if there is any efficiently computable way to spot a pattern in mathematics, and the market’s prices fail to reflect that pattern, then a trader can use the pattern to exploit the market, so a logical inductor must come to reflect every such pattern. The reason this is not an empty wish is that such markets demonstrably exist, and exist as something a computer can run.

Theorem 4.4 (Existence, Garrabrant et al.). *For any deductive process, there is a market that a computer can generate and that satisfies the logical induction criterion relative to that process. In particular, for any reasonable mathematical theory (any theory whose theorems a program can enumerate), there is a computable logical inductor whose prices run ahead of that theory.*

5 How such a market is built

The existence proof is constructive, and its mechanism is worth sketching, because it shows why “no efficient trader can win” is achievable rather than merely desirable. There are two moves.

The first handles a *single* trader. Because a trader’s orders are expressible features and so vary continuously with the prices, and the market gets to choose those prices, the market can always find prices at which a given trader is left with no profitable move, the “fair prices” at which it neither buys nor sells on net: nudge a price up when the trader is a net buyer of that share and down when it is a net seller, and a balance point exists. The mathematical guarantee that such a balancing point exists is a classic result called Brouwer’s fixed point theorem, which says that any continuous way of nudging the points of a suitable region back into itself must leave at least one point unmoved; here the unmoved point is the fair-price vector.

The second move handles *all* efficient traders at once by bundling them into a single one, which is possible precisely because trading strategies are affine combinations and affine combinations can simply be added.

Enumerate the efficiently computable traders (restricting to those that risk at most one dollar in the worst case) and build one master “trading firm” that, on day n , runs the first n of them in parallel, allotting the i th trader a budget of 2^{-i} of the whole, and summing their strategies into one combined strategy. This firm beats the market if and only if some individual efficient trader does: if trader number k could win without bound, the firm captures a 2^{-k} share of that still-unbounded windfall, while every other trader it runs can cost it at most its tiny allotment, so the firm’s total possible loss is capped at $\sum_i 2^{-i} = 1$ dollar. The market maker then sets its prices, using the single-trader fair-price trick applied to this combined firm, so that the firm earns at most 2^{-n} on day n . The firm’s lifetime earnings are then capped at $\sum_n 2^{-n} = 1$ dollar, a finite amount. Since the firm was the most dangerous trader available, dominating every individual efficient trader, and even it cannot make unbounded profit, no efficient trader can exploit the market. The price search is a brute-force scan over candidate prices, so the whole construction is computable, and the resulting reasoner has a finite belief state every day, the spreadsheet image made literal. The cost of this generality is efficiency: the construction is wildly impractical to actually run, but its mere existence is what licenses every property in the next section.

6 What the single criterion forces

The reason logical induction is celebrated is that the one criterion of Definition 4.1 entails a long list of the properties one would want from a reasoner under logical uncertainty. Each is proved the same way, by exhibiting the efficiently computable trader that would exploit any market lacking the property; since a logical inductor admits no such trader, it must have the property. We survey the main ones, and the recurring move is always the line from Section 4: if the prices failed to do this, here is the fast trader that would pump them.

The prices must settle down, and settle on coherent probabilities. If a statement’s price oscillated forever between, say, thirty cents and seventy cents, a trader could buy low and sell high across the swings indefinitely, an unlimited profit off a limited stake; so every statement’s price approaches a limit. Those limiting prices form a genuine, logically consistent probability assignment: provable statements head to certainty, refutable ones to zero, and the probabilities of mutually exclusive exhaustive alternatives sum to one. (If a provable statement settled at ninety-nine cents, a trader buying at that price and collecting the eventual dollar would skim a penny forever.) This is what retroactively earns the prices the name “probabilities.”

The reasoner learns quickly, faster than it can prove things. Coherence eventually is cheap; the powerful property is timeliness. For any efficiently describable sequence of theorems, the reasoner comes to price the n th one near certainty by day n , no matter how astronomically long its actual proof is. Were it to lag, pricing the n th theorem below certainty on day n , the trader who recognizes the cheap-to-describe pattern generating the sequence and buys that share on day n would profit indefinitely. Competitive pressure forces the prices to anticipate the pattern long before the slow deductive process grinds out the proofs.

The reasoner is well calibrated and learns the right frequencies. Among the statements it prices at some level p , about a p fraction come true. And when a sequence of statements is effectively unpredictable to anything as fast as the reasoner, with no efficient way to forecast which are true, the reasoner settles on the correct underlying frequency. The flagship case is the one from the wish list: it comes to price “the n th digit of π is a seven” near ten percent for large n , because any systematic deviation from ten percent would be a pattern a trader could exploit, while the digits themselves are too hard to predict for it to do any better than the base rate.

The reasoner respects relationships among statements it cannot individually evaluate. For an efficiently described stream of pairs of statements that are mutually exclusive and exhaustive, it brings each pair’s two prices to sum to about one in a timely way, even when neither member’s truth is known. It will price “the 173,498th digit of π is even” and “... is odd” so that the two sum to nearly one long before anyone computes that digit, because any gap from one is an arbitrage and arbitrages get punished.

The reasoner never grows dogmatic without proof. A statement the process never refutes keeps a limiting probability bounded above zero; one it never proves keeps a limiting probability bounded below one. (The trader enforcing this buys a never-refuted statement at successively halved prices, fifty cents, then twenty-five, then twelve and a half, banking unbounded value should the price ever reach zero.) On any particular day the reasoner can be pushed to a wrong extreme, but in the long run it keeps an open mind about

everything it has not actually settled.

Finally, the reasoner comes to know, and trust, its own beliefs. Because the reasoner’s own prices are themselves generated by a program, hence form a pattern, the reasoner comes to price statements about its own prices accurately, a kind of self-knowledge. The most striking instance, and the one that ties this material to the tiling-agents problem, is self-trust: a logical inductor’s current probability for a statement is, to good approximation, a weighted average of what it expects its own *future* probabilities for that statement to be. It defers to its later, better-informed self without having to watch the reasoning its future self will do. This is a probabilistic cousin of exactly the trust a parent agent wants in a successor whose specific conclusions it cannot anticipate, and it is notable that here the deference falls out for free from unexploitability, where in the proof-based setting of tiling agents the analogous self-trust collides with Löb’s theorem.

7 Connections and open problems

It helps to place logical induction against the coherence theorems it grows out of, and to be honest about what it does not yet do.

The lineage runs straight through the Dutch book. A Dutch book, recall, is a bundle of bets rigged so that the bettor loses no matter what happens, and the classical argument grounding probability says a rational agent’s beliefs must be free of Dutch books: no system of bets can be arranged to take its money for sure. That argument quietly assumes logical omniscience, because verifying that a bundle of bets is a guaranteed loss requires knowing all the logical relationships among the bets, which is exactly the computation a bounded reasoner cannot afford. Logical induction keeps the spirit of the Dutch book and weakens its letter precisely where the omniscience was hiding. Instead of demanding that *no* exploiting bundle exist, it demands only that no *efficiently computable* exploiting bundle exist. That is a weaker requirement, one a real computer can meet, yet strong enough to recover full coherence in the limit, so it sits beneath classical Bayesian coherence as the computationally realistic foundation the coherence theorems wanted but could not have. The move is the same one that runs through cryptography and learning theory, from “no adversary can break it” to “no efficient adversary can break it.”

There is also a family resemblance to Solomonoff induction, an idealized theory of learning from empirical observation, which holds all computable hypotheses as competing experts and reweights them by predictive accuracy. Logical induction holds all efficiently computable traders as experts and reweights them, in effect, by trading profit, with the market price emerging as their aggregate verdict. The difference is that Solomonoff’s experts each model the entire world, while logical induction’s traders may specialize, each watching one corner of mathematics, which is what lets the market knit partial competences into a single global belief state. The machinery is not even tied to mathematics: feed it a stream of pixels instead of a stream of theorems and it learns empirical patterns, so it is best seen as a general theory of timely, pattern-respecting prediction that we happen to be pointing at the deductive stream.

Two limitations mark the live frontier. A logical inductor has no notion of *deciding what to think about*: it improves its beliefs about everything in parallel, with no way to aim its computation at the one question that currently matters, the way a person chooses which problem to work on. And it cannot reason about logical *impossibilities*. “If Fermat’s Last Theorem were false, then. . .” is something mathematicians do productively, but a logical inductor eventually prices that falsehood at zero, and conditioning on a probability-zero event is undefined, so coherent reasoning about impossible mathematical scenarios stays out of reach. Both gaps matter for decision-making, where an agent embedded in the world must reason about logically-structured counterfactuals (what a perfect predictor will have computed, what another copy of its own procedure will output) under exactly the kind of logical uncertainty this theory formalizes. Logical induction is the first framework that lets a computable, embedded reasoner be sensibly uncertain about mathematics and grow steadily, provably less wrong; making that reasoner also *decide* well is the next problem.

Sources

- *An Intuitive Guide to Garrabrant Induction* (read in full). <https://www.lesswrong.com/posts/y5GftLezdozEHdXkL/an-intuitive-guide-to-garrabrant-induction>

- Garrabrant, Benson-Tilsen, Critch, Soares, and Taylor, *Logical Induction* (Chapters 1 and 3 in detail: the desiderata, and the market, deductive-process, trader, exploitation definitions and the existence theorem; Chapter 4 skimmed for the properties of logical inductors). <https://arxiv.org/pdf/1609.03543>